```
Lab1- Data colecting
# Requests allows us to make HTTP requests which we will use to get data
from an API
import requests
# Pandas is a software library written for the Python programming language
for data manipulation and analysis.
import pandas as pd
# NumPy is a library for the Python programming language, adding support
for large, multi-dimensional arrays and matrices, along with a large
collection of high-level mathematical functions to operate on these arrays
import numpy as np
# Datetime is a library that allows us to represent dates
import datetime

# Setting this option will print all collumns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)


# Takes the dataset and uses the rocket column to call the API and append
the data to the list
def getBoosterVersion(data):
    for x in data['rocket']:
       if x:
        response =
requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
        BoosterVersion.append(response['name'])


# Takes the dataset and uses the launchpad column to call the API and
append the data to the list
def getLaunchSite(data):
    for x in data['launchpad']:
       if x:
         response =
requests.get("https://api.spacexdata.com/v4/launchpads/"+str(x)).json()
         Longitude.append(response['longitude'])
         Latitude.append(response['latitude'])
         LaunchSite.append(response['name'])
# Takes the dataset and uses the payloads column to call the API and append
the data to the lists
def getPayloadData(data):
    for load in data['payloads']:
       if load:
        response =
requests.get("https://api.spacexdata.com/v4/payloads/"+load).json()
        PayloadMass.append(response['mass_kg'])
        Orbit.append(response['orbit'])
# Takes the dataset and uses the cores column to call the API and append
the data to the lists
def getCoreData(data):
    for core in data['cores']:
            if core['core'] != None:
                response =
requests.get("https://api.spacexdata.com/v4/cores/"+core['core']).json()
                Block.append(response['block'])
```

```
                ReusedCount.append(response['reuse_count'])
                Serial.append(response['serial'])
            else:
                Block.append(None)
                ReusedCount.append(None)
                Serial.append(None)
            Outcome.append(str(core['landing_success'])+'
'+str(core['landing_type']))
            Flights.append(core['flight'])
            GridFins.append(core['gridfins'])
            Reused.append(core['reused'])
            Legs.append(core['legs'])
            LandingPad.append(core['landpad'])

spacex_url="https://api.spacexdata.com/v4/launches/past"


response = requests.get(spacex_url)


print(response.content)


static_json_url='https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/API_call_spacex_api.json'


response=requests.get(static_json_url)


response.status_code


# Use json_normalize meethod to convert the json result into a dataframe
data = response.json() # Décodage du contenu de la réponse
df = pd.json_normalize(data)


# Get the head of the dataframe
print(df.head())


#import pandas as pd

# 1. Assurez-vous que 'data' est converti en DataFrame
data = pd.DataFrame(data)

# 2. Prenez le sous-ensemble des colonnes que vous voulez
features = ['rocket', 'payloads', 'launchpad', 'cores', 'flight_number',
'date_utc']
data_subset = data[features].copy()

# 3. Ensuite, appliquez votre filtre (ex: exactement 1 élément dans 'cores'
et 'payloads')
data_filtered = data_subset[
    (data_subset['cores'].apply(len) == 1) &
    (data_subset['payloads'].apply(len) == 1)
```

```
]
# We also want to convert the date_utc to a datetime datatype and then
extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

#Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]


#import pandas as pd

# 1. Assurez-vous que 'data' est converti en DataFrame
data = pd.DataFrame(data)

# 2. Prenez le sous-ensemble des colonnes que vous voulez
features = ['rocket', 'payloads', 'launchpad', 'cores', 'flight_number',
'date_utc']
data_subset = data[features].copy()

# 3. Ensuite, appliquez votre filtre (ex: exactement 1 élément dans 'cores'
et 'payloads')
data_filtered = data_subset[
    (data_subset['cores'].apply(len) == 1) &
    (data_subset['payloads'].apply(len) == 1)
]
# We also want to convert the date_utc to a datetime datatype and then
extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

#Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]


#Global variables
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []


BoosterVersion


#import requests

def getBoosterVersion(data):
    for x in data['rocket']:
```

```
        if x:
            # Complete your URL here (e.g., 'https://spacexdata.com' +
str(x))
            url = f"https://spacexdata.com{x}"
            response = requests.get(url)

            # Check if the request was successful
            if response.status_code == 200:
                try:
                    # Attempt to parse JSON
                    BoosterVersion.append(response.json()['name'])
                except requests.exceptions.JSONDecodeError:
                    print(f"Error decoding JSON for {x}. Response text:
{response.text}")
                    BoosterVersion.append(None)
            else:
                print(f"Failed to retrieve data for {x}. Status code:
{response.status_code}")
                BoosterVersion.append(None)


# Take a look at the first 5 elements to verify
BoosterVersion[0:5]


#import requests

def getLaunchSite(data):
    for x in data.get('launchpad', []):
        if x:
            url = f"https://api.spacexdata.com/v4/launchpads/{str(x)}"
            response = requests.get(url)

            if response.ok: # Checks if HTTP status code is 200-299
                launchpad_data = response.json()
                # Extract and use your data here, e.g.:
                # longitude = launchpad_data['longitude']
                # latitude = launchpad_data['latitude']
                print(f"Success for pad {x}")
            else:
                print(f"Failed to retrieve data for {x}. Status:
{response.status_code}")


#import requests

def getLaunchSite(data):
    for x in data.get('launchpad', []):
        if x:
            url = f"https://api.spacexdata.com/v4/launchpads/{str(x)}"
            response = requests.get(url)

            if response.ok: # Checks if HTTP status code is 200-299
                launchpad_data = response.json()
                # Extract and use your data here, e.g.:
                # longitude = launchpad_data['longitude']
                # latitude = launchpad_data['latitude']
```

```
                print(f"Success for pad {x}")
            else:
                print(f"Failed to retrieve data for {x}. Status:
{response.status_code}")


# Call getLaunchSite
#getLaunchSite(data)


for load_item in data['payloads']:
    # Ensure it's not empty/null
    if load_item:
        # Check if the payload is a list (nested payload info) or a single
payload ID string
        if isinstance(load_item, list):
            for payload_id in load_item:
                url =
f"https://api.spacexdata.com/v4/payloads/{payload_id}"
                response = requests.get(url)

                # Verify the request was successful before parsing JSON
                if response.status_code == 200:
                    try:
                        payload_data = response.json()
                        PayloadMass.append(payload_data.get('mass_kg'))
                        Orbit.append(payload_data.get('orbit'))
                    except ValueError:
                        print(f"Failed to decode JSON for payload ID:
{payload_id}")
                else:
                    print(f"API Error {response.status_code} for payload
ID: {payload_id}")
                    PayloadMass.append(None)
                    Orbit.append(None)
        else:
            # If load_item is already the payload ID string
            url = f"https://api.spacexdata.com/v4/payloads/{load_item}"
            response = requests.get(url)

            if response.status_code == 200:
                try:
                    payload_data = response.json()
                    PayloadMass.append(payload_data.get('mass_kg'))
                    Orbit.append(payload_data.get('orbit'))
except ValueError:
                    print(f"Failed to decode JSON for payload ID:
{load_item}")
            else:
                print(f"API Error {response.status_code} for payload ID:
{load_item}")
                PayloadMass.append(None)
                Orbit.append(None)
# Call getPayloadData
#getPayloadData(data)
```

```
#import requests

def getCoreData(data, Block):
    for core in data['cores']:
        # Vérifie si core n'est pas None (l'identifiant existe)
        if core is not None:
            # On utilise directement core, car c'est déjà l'identifiant
(ex: "B1051")
            url = f"https://api.spacexdata.com/v4/cores/{core}"
            response = requests.get(url).json()
            Block.append(response['block'])

# Exemple d'initialisation de votre liste
# Block = []
# getCoreData(data, Block)


# Call getCoreData
#getCoreData(data)


launch_dict = {'FlightNumber': list(data['flight_number']),
'Date': list(data['date']),
'BoosterVersion':BoosterVersion,
'PayloadMass':PayloadMass,
'Orbit':Orbit,
'LaunchSite':LaunchSite,
'Outcome':Outcome,
'Flights':Flights,
'GridFins':GridFins,
'Reused':Reused,
'Legs':Legs,
'LandingPad':LandingPad,
'Block':Block,
'ReusedCount':ReusedCount,
'Serial':Serial,
'Longitude': Longitude,
'Latitude': Latitude}


#import pandas as pd

# Correction : utiliser json_normalize si les longueurs ou les imbrications
diffèrent
df = pd.json_normalize(launch_dict)

# Display the first 5 rows to verify your data
print(df.head())


# Show the head of the dataframe

# 1. Display the structural summary (data types, non-null counts)
print("--- DataFrame Information ---")
print(df.info())

# 2. Display the statistical summary (mean, min, max, percentiles for
numerical columns)
```

```
print("\n--- Statistical Summary ---")
print(df.describe())


# Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = df[df['BoosterVersion'] != 'Falcon 1']


data_falcon9.loc[:,'FlightNumber'] = list(range(1,
data_falcon9.shape[0]+1))
data_falcon9


# Afficher le décompte des valeurs manquantes par colonne
print(data_falcon9.isnull().sum())


print(data_falcon9.columns)


#import numpy as np
#import pandas as pd

# 1. Convertir la colonne en valeurs numériques (les erreurs deviennent
NaN)
data_falcon9['PayloadMass'] = pd.to_numeric(data_falcon9['PayloadMass'],
errors='coerce')

# 2. Calculer la moyenne de la colonne PayloadMass
payload_mass_mean = data_falcon9['PayloadMass'].mean()

# 3. Remplacer les valeurs NaN par la moyenne calculée
data_falcon9['PayloadMass'] =
data_falcon9['PayloadMass'].fillna(payload_mass_mean)
```

Web scraping Falcon 9 and Falcon Heavy Launches Records from Wikipedia

```
!pip3 install beautifulsoup4
!pip3 install requests
!pip install pandas


import sys

import requests
from bs4 import BeautifulSoup
import re
import unicodedata
import pandas as pd


def date_time(table_cells):
    """
    This function returns the data and time from the HTML  table cell
```

```
    Input: the  element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in
list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
    This function returns the booster version from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=''.join([booster_version for i,booster_version in enumerate(
table_cells.strings) if i%2==0][0:-1])
    return out

def landing_status(table_cells):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=[i for i in table_cells.strings][0]
    return out


def get_mass(table_cells):
    mass=unicodedata.normalize("NFKD", table_cells.text).strip()


if mass:
        mass.find("kg")
        new_mass=mass[0:mass.find("kg")+2]
    else:
        new_mass=0
    return new_mass


def extract_column_from_header(row):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    if (row.br):
        row.br.extract()
    if row.a:
        row.a.extract()
    if row.sup:
        row.sup.extract()

    colunm_name = ' '.join(row.contents)

    # Filter the digit and empty names
    if not(colunm_name.strip().isdigit()):
        colunm_name = colunm_name.strip()
        return colunm_name
```

```
static_url =
"https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Hea
vy_launches&oldid=1027686922"

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) "
                  "Chrome/91.0.4472.124 Safari/537.36"
}


# use requests.get() method with the provided static_url and headers
# assign the response to a object

# Assurez-vous que static_url et headers sont définis au préalable
response = requests.get(static_url, headers=headers)

# Affichage du code de statut pour vérifier le succès de la requête
print("Code de statut :", response.status_code)


# Use BeautifulSoup() to create a BeautifulSoup object from a response text
content

soup = BeautifulSoup(response.text, 'html.parser')


# Use soup.title attribute

print("Page Title:", soup.title.string)


# Use the find_all function in the BeautifulSoup object, with element type
`table`
# Assign the result to a list called `html_tables`

# Recherche de tous les tableaux de la page
html_tables = soup.find_all('table')

# Ciblez le tableau contenant les enregistrements de lancement
(généralement le 3ème tableau, soit l'index 2)
# Vous pouvez ajuster cet index selon la structure exacte de la page wiki
récupérée
#first_launch_table = html_tables[2]


# Let's print the third table and check its content
first_launch_table = html_tables[2]
print(first_launch_table)


column_names = []

# Apply find_all() function with `th` element on first_launch_table
# Iterate each th element and apply the provided
extract_column_from_header() to get a column name
# Append the Non-empty column name (`if name is not None and len(name) >
0`) into a list called column_names
```

```
# Extraction des noms de colonnes depuis les éléments <th>
column_names = []

# Fonction pour extraire et nettoyer le texte de l'en-tête (si fournie dans
votre TP)
def extract_column_from_header(row):
    if (row.br):
        row.br.extract()
    if (row.a):
        row.a.extract()
    if (row.sup):
        row.sup.extract()

    column_name = ' '.join(row.contents)
    if(column_name is not None and len(column_name) > 0):
        return column_name.strip()
    return None

# Itération sur les lignes d'en-tête (<th>)
for row in first_launch_table.find_all('th'):
    name = extract_column_from_header(row)
    if name is not None and len(name) > 0:
        column_names.append(name)


print(column_names)


launch_dict= dict.fromkeys(column_names)

# Remove an irrelvant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]


extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable
plainrowheaders collapsible")):
   # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to
launch a number
        if rows.th:
```

```
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
        else:
            flag=False
        #get table element
        row=rows.find_all('td')
        #if it is number save cells in a dictonary
        if flag:
            extracted_row += 1
            # Flight Number value
            # TODO: Append the flight_number into launch_dict with key
`Flight No.`
            launch_dict['Flight No.'].append(flight_number)
            #print(flight_number)
            datatimelist=date_time(row[0])

            # Date value
            # TODO: Append the date into launch_dict with key `Date`
            #date = datatimelist[0].strip(',')
date = datatimelist[0].strip()
            launch_dict['Date'].append(date)
            #print(date)

            # Time value
            # TODO: Append the time into launch_dict with key `Time`
            time = datatimelist[1]

            launch_dict['Time'].append(time)
            #print(time)

            # Booster version
            # TODO: Append the bv into launch_dict with key `Version
Booster`
            bv = booster_version(row[1])
            if not(bv):
                bv = row[1].a.string.strip() if row[1].a else ""
            launch_dict['Version Booster'].append(bv)
             #bv=booster_version(row[1])
            #if not(bv):
               # bv=row[1].a.string
            print(bv)

            # Launch Site
            # TODO: Append the bv into launch_dict with key `Launch Site`

            launch_site = row[2].a.string if row[2].a else ""
            launch_dict['Launch site'].append(launch_site)

            #launch_site = row[2].a.string
            #print(launch_site)
# Payload
            # TODO: Append the payload into launch_dict with key `Payload`
            payload = row[3].a.string.strip() if row[3].a else ""
            launch_dict['Payload'].append(payload)

            #payload = row[3].a.string
```

```
            #print(payload)

            # Payload Mass
            # TODO: Append the payload_mass into launch_dict with key
`Payload mass`
            payload_mass = get_mass(row[4])
            launch_dict['Payload mass'].append(payload_mass)

            #payload_mass = get_mass(row[4])
            #print(payload)

            # Orbit
            # TODO: Append the orbit into launch_dict with key `Orbit`
            orbit = row[5].a.string.strip() if row[5].a else ""
            launch_dict['Orbit'].append(orbit)

            #orbit = row[5].a.string
            #print(orbit)

            # Customer
            # TODO: Append the customer into launch_dict with key
`Customer`
            customer = row[6].a.string if row[6].a else ""
            launch_dict['Customer'].append(customer)
            #customer = row[6].a.string
            #print(customer)
# Launch outcome
            # TODO: Append the launch_outcome into launch_dict with key
`Launch outcome`
            launch_outcome = list(row[7].strings)[0].strip() if row[7] else
""
            launch_dict['Launch outcome'].append(launch_outcome)

            #launch_outcome = list(row[7].strings)[0]
            #print(launch_outcome)

            # Booster landing
            # TODO: Append the launch_outcome into launch_dict with key
`Booster landing`
            booster_landing = list(row[8].strings)[0].strip() if row[8]
else ""
            launch_dict['Booster landing'].append(booster_landing)

            #booster_landing = landing_status(row[8])
            #print(booster_landing)


df= pd.DataFrame({ key:pd.Series(value) for key, value in
launch_dict.items() })
```

Lab 2: Data wrangling

```
!pip install pandas
!pip install numpy
```

```python
# Pandas is a software library written for the Python programming language
for data manipulation and analysis.
import pandas as pd
#NumPy is a library for the Python programming language, adding support for
large, multi-dimensional arrays and matrices, along with a large collection
of high-level mathematical functions to operate on these arrays
import numpy as np


df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/dataset_part_1.csv")
df.head(10)


df.isnull().sum()/len(df)*100


df.dtypes


# Apply value_counts() on column LaunchSite
df['LaunchSite'].value_counts()


# Apply value_counts on Orbit column

# 1. Filtrer les données pour exclure les orbites 'GTO'
df_filtered = df[df['Orbit'] != 'GTO']

# 2. Calculer le nombre d'apparitions (fréquence absolue)
nombre_orbites = df_filtered['Orbit'].value_counts()

# 3. Calculer la fréquence d'apparition (fréquence relative / pourcentage)
frequence_orbites = df_filtered['Orbit'].value_counts(normalize=True)

# Affichage des résultats
print("Nombre d'apparitions de chaque orbite :")
print(nombre_orbites)
print("\nFréquence d'apparition de chaque orbite (en %) :")
print(frequence_orbites * 100)


# landing_outcomes = values on Outcome column

# Calcule le nombre et la fréquence des résultats d'atterrissage
landing_outcomes = df['Outcome'].value_counts()

# Afficher les résultats
print(landing_outcomes)


for i,outcome in enumerate(landing_outcomes.keys()):
    print(i,outcome)


bad_outcomes=set(landing_outcomes.keys()[[1,3,5,6,7]])
bad_outcomes
```

```
# landing_class = 0 if bad_outcome
# landing_class = 1 otherwise

landing_class = [0 if outcome in bad_outcomes else 1 for outcome in
df['Outcome']]


df['Class']=landing_class
df[['Class']].head(8)


df.head(5)


df["Class"].mean()


df.to_csv("dataset_part_2.csv", index=False)
```

Hands-on Lab: Complete the EDA with Visualization

```
!pip install pandas
!pip install numpy
!pip install seaborn
!pip install matplotlib


df=pd.read_csv("https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/dataset_part_2.csv")

# If you were unable to complete the previous lab correctly you can
uncomment and load this csv

# df = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DS0701EN-
SkillsNetwork/api/dataset_part_2.csv')

df.head(5)


sns.catplot(y="PayloadMass", x="FlightNumber", hue="Class", data=df, aspect
= 5)
plt.xlabel("Flight Number",fontsize=20)
plt.ylabel("Pay load Mass (kg)",fontsize=20)
plt.show()


# Plot a scatter point chart with x axis to be Flight Number and y axis to
be the launch site, and hue to be the class value
#import matplotlib.pyplot as plt
#import seaborn as sns

# Code pour tracer la relation entre Flight Number et Launch Site
sns.catplot(x="FlightNumber", y="LaunchSite", hue="Class", data=df,
aspect=5)
```

```python
# Configuration des étiquettes des axes
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)

# Affichage du graphique
plt.show()


# Plot a scatter point chart with x axis to be Pay Load Mass (kg) and y axis to be the launch site, and hue to be the class value

# -- TASK 2: Relation entre la masse de la charge utile (Payload) et le site de lancement --
# Graphique en nuage de points (Scatter chart) : Charge utile vs Site de lancement
sns.catplot(
    x="PayloadMass",  # Axe X représentant la masse de la charge utile en kg
    y="LaunchSite",  # Axe Y représentant le site de lancement
    hue="Class",  # Coloration selon le résultat (0 = Échec, 1 = Succès)
    data=df,  # Votre DataFrame pandas contenant les données de SpaceX
    aspect=5,  # Ajustement de la largeur du graphique pour une meilleure lisibilité
)

# Configuration des étiquettes et affichage du graphique
plt.xlabel("Payload Mass (kg)", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)
plt.title("Relationship between Payload Mass and Launch Site", fontsize=20)
plt.show()


# -- Complément : FlightNumber vs LaunchSite --
# Graphique demandé dans votre description textuelle
sns.catplot(
    x="FlightNumber",  # Axe X représentant le numéro de vol
    y="LaunchSite",  # Axe Y représentant le site de lancement
    hue="Class",  # Coloration selon la classe de succès
    data=df,
    aspect=5,
)

# Configuration des étiquettes et affichage
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Launch Site", fontsize=20)
plt.title("Flight Number vs Launch Site", fontsize=20)
plt.show()


# HINT use groupby method on Orbit column and get the mean of Class column

# 1. Calculer le taux de succès moyen pour chaque type d'orbite
# Le reset_index() transforme le résultat de l'agrégation en un DataFrame propre
df_orbit = df.groupby('Orbit')['Class'].mean().reset_index()
```

```
# 2. Trier par taux de succès décroissant pour une meilleure lisibilité
df_orbit = df_orbit.sort_values(by='Class', ascending=False)

# 3. Créer le graphique à barres
plt.figure(figsize=(10, 6))
sns.barplot(x='Orbit', y='Class', data=df_orbit, palette='Blues_r')

# 4. Personnaliser les étiquettes et le titre
plt.title('Success Rate of Each Orbit Type', fontsize=16)
plt.xlabel('Orbit Type', fontsize=14)
plt.ylabel('Success Rate (Mean of Class)', fontsize=14)

# 5. Fixer la limite de l'axe Y de 0 à 1 (représentant 0% à 100%)
plt.ylim(0, 1)
# 6. Afficher le graphique
plt.show()


# Plot a scatter point chart with x axis to be FlightNumber and y axis to
be the Orbit, and hue to be the class value

# TASK 4: Visualiser la relation entre le numéro de vol (FlightNumber) et
le type d'orbite (Orbit)
# Tracé d'un graphique en nuage de points (scatter/categorical plot)
sns.catplot(
    x="FlightNumber",
    y="Orbit",
    hue="Class",       # La couleur varie selon la valeur de la classe (0 ou
1)
    data=df,           # Remplacez 'df' par le nom de votre DataFrame si
nécessaire
    aspect=5           # Ajuste la largeur pour mieux étaler les points le
long de l'axe X
)

# Configuration des étiquettes des axes
plt.xlabel("Flight Number", fontsize=20)
plt.ylabel("Orbit Type", fontsize=20)
plt.title("Relationship between Flight Number and Orbit Type", fontsize=16)

# Affichage du graphique
plt.show()



# Plot a scatter point chart with x axis to be Payload and y axis to be the
Orbit, and hue to be the class value

# TASK 5: Visualisation de la relation entre la masse de la charge utile
(Payload) et le type d'orbite
# Graphique de type nuage de points (scatter plot)
# Axe X : PayloadMass, Axe Y : Orbit, Couleur (hue) : Class
sns.catplot(
    x="PayloadMass",
    y="Orbit",
    hue="Class",
```

```
    data=df,
    aspect=1.5,
    height=6
)

# Configuration des étiquettes des axes et du titre
plt.xlabel("Masse de la charge utile (Payload Mass en kg)", fontsize=12)
plt.ylabel("Type d'Orbite (Orbit)", fontsize=12)
plt.title("Relation entre la Charge Utile et le Type d'Orbite en fonction
du succès", fontsize=14)

# Affichage du graphique
plt.show()


# A function to Extract years from the date
year=[]
def Extract_year(date):
    for i in df["Date"]:
        year.append(i.split("-")[0])
    return year



import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

# 1. Définition de la fonction pour extraire l'année
year = []


def Extract_year():
    for i in df["Date"]:
        year.append(str(i).split("-")[0])
    return year


# 2. Application de la fonction pour créer la colonne 'Year'
# On réinitialise la liste 'year' au cas où le code est exécuté plusieurs
fois
year = []
Extract_year()
df["Year"] = year

# 3. Calcul du taux moyen de succès par année
# La colonne du taux de succès s'appelle généralement 'Class' dans ce
dataset
df_yearly_success = df.groupby("Year")["Class"].mean().reset_index()

# 4. Tracer le graphique linéaire de la tendance annuelle
plt.figure(figsize=(10, 6))
sns.lineplot(data=df_yearly_success, x="Year", y="Class", marker="o")

# 5. Personnalisation des axes et du titre
plt.title("Launch Success Yearly Trend", fontsize=16)
plt.xlabel("Year", fontsize=12)
```

```
plt.ylabel("Average Success Rate", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.6)

# Affichage du graphique
plt.show()
```

```
features = df[['FlightNumber', 'PayloadMass', 'Orbit', 'LaunchSite',
'Flights', 'GridFins', 'Reused', 'Legs', 'LandingPad', 'Block',
'ReusedCount', 'Serial']]
features.head()
```

```
# HINT: Use get_dummies() function on the categorical columns

import pandas as pd

# Application du OneHotEncoder avec get_dummies sur les colonnes spécifiées
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'LaunchSite',
'LandingPad', 'Serial'])

# Affichage des premières lignes du DataFrame résultant
features_one_hot.head()
```

```
# HINT: use astype function
# Cast the entire DataFrame to float64
features_one_hot = features_one_hot.astype('float64')
```

Assignment: SQL Notebook for Peer Assignment

```
!pip install sqlalchemy==1.3.9
```

```
!pip install ipython-sql
!pip install ipython-sql prettytable
```

```
%load_ext sql
```

```
import csv, sqlite3
import prettytable
prettytable.DEFAULT = 'DEFAULT'

con = sqlite3.connect("my_data1.db")
cur = con.cursor()
```

```
!pip install -q pandas
```

```
%sql sqlite:///my_data1.db
```

```
import pandas as pd
df = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/labs/module_2/data/Spacex.csv")
df.to_sql("SPACEXTBL", con, if_exists='replace',
index=False,method="multi")


#DROP THE TABLE IF EXISTS

%sql DROP TABLE IF EXISTS SPACEXTABLE;


%sql create table SPACEXTABLE as select * from SPACEXTBL where Date is not
null


%%sql
SELECT DISTINCT "Launch_Site" FROM SPACEXTBL;


%sql SELECT * FROM SPACEXTBL WHERE LAUNCH_SITE LIKE 'CCA%' LIMIT 5;


%sql SELECT SUM(PAYLOAD_MASS__KG_) AS "Total Payload Mass" FROM SPACEXTBL
WHERE CUSTOMER = 'NASA (CRS)';


# Calcul et affichage de la masse moyenne de la charge utile pour la
version F9 v1.1
avg_payload = df[df["Booster_Version"] == 'F9
v1.1']["PAYLOAD_MASS__KG_"].mean()

print(f"Average payload mass carried by booster version F9 v1.1:
{avg_payload} kg")


%sql SELECT MIN(Date) FROM SPACEXTABLE WHERE Landing_Outcome = 'Success
(ground pad)'


print("Boosters avec succès sur drone ship (Masse entre 4000 et 6000 kg)
:")

# Filtrer le DataFrame selon les critères demandés
boosters_filtres = df[
    (df["Landing_Outcome"].str.contains('Success (drone ship)', case=False,
na=False, regex=False)) &
    (df["PAYLOAD_MASS__KG_"] > 4000) &
    (df["PAYLOAD_MASS__KG_"] < 6000)
]

# Lister les noms uniques des versions de boosters
print(boosters_filtres["Booster_Version"].unique())


%sql SELECT MISSION_OUTCOME, COUNT(*) AS TOTAL_NUMBER FROM SPACEXTBL GROUP
BY MISSION_OUTCOME;
```

```
%sql SELECT BOOSTER_VERSION FROM SPACEXTBL WHERE PAYLOAD_MASS__KG_ =
(SELECT MAX(PAYLOAD_MASS__KG_) FROM SPACEXTBL);
```

```
%%sql
SELECT DISTINCT
    CASE substr(DATE, 6, 2)
        WHEN '01' THEN 'January' WHEN '02' THEN 'February'
        WHEN '03' THEN 'March' WHEN '04' THEN 'April'
        WHEN '05' THEN 'May' WHEN '06' THEN 'June'
        WHEN '07' THEN 'July' WHEN '08' THEN 'August'
        WHEN '09' THEN 'September' WHEN '10' THEN 'October'
        WHEN '11' THEN 'November' WHEN '12' THEN 'December'
    END AS Month,
    LANDING_OUTCOME, BOOSTER_VERSION, LAUNCH_SITE
FROM SPACEXTBL
WHERE Landing_Outcome = 'Failure (drone ship)'
  AND substr(DATE, 1, 4) = '2015';
```

```
import pandas as pd

# 1. Chargement de votre jeu de données (remplacez par votre source si
nécessaire)
# df = pd.read_csv("spacex_web_scraped.csv")

# 2. S'assurer que la colonne Date est au format datetime
df['Date'] = pd.to_datetime(df['Date'])

# 3. Filtrer les données entre les deux dates cibles (inclusivement)
filtered_df = df[(df['Date'] >= '2010-06-04') & (df['Date'] <= '2017-03-
20')]

# 4. Compter et classer les résultats d'atterrissage par ordre décroissant
# Note: Remplacez 'Landing_Outcome' par le nom exact de votre colonne (ex:
'LandingOutcome' ou 'Outcome')
landing_counts =
filtered_df["Landing_Outcome"].value_counts().reset_index()
landing_counts.columns = ["Landing_Outcome", 'Count']

# Affichage du classement
print(landing_counts)
```

Launch Sites Locations Analysis with Folium

```
!pip3 install folium
!pip3 install wget
!pip3 install pandas
```

```
import folium
import wget
import pandas as pd
```

```
# Import folium MarkerCluster plugin
from folium.plugins import MarkerCluster
# Import folium MousePosition plugin
from folium.plugins import MousePosition
# Import folium DivIcon plugin
from folium.features import DivIcon


# Download and read the `spacex_launch_geo.csv`
spacex_csv_file = wget.download('https://cf-courses-data.s3.us.cloud-
object-storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/spacex_launch_geo.csv')
spacex_df=pd.read_csv(spacex_csv_file)


# Select relevant sub-columns: `Launch Site`, `Lat(Latitude)`,
`Long(Longitude)`, `class`
spacex_df = spacex_df[['Launch Site', 'Lat', 'Long', 'class']]
launch_sites_df = spacex_df.groupby(['Launch Site'],
as_index=False).first()
launch_sites_df = launch_sites_df[['Launch Site', 'Lat', 'Long']]
launch_sites_df


# Start location is NASA Johnson Space Center
nasa_coordinate = [29.559684888503615, -95.0830971930759]
site_map = folium.Map(location=nasa_coordinate, zoom_start=10)


# Create a blue circle at NASA Johnson Space Center's coordinate with a
popup label showing its name
circle = folium.Circle(nasa_coordinate, radius=1000, color='#d35400',
fill=True).add_child(folium.Popup('NASA Johnson Space Center'))
# Create a blue circle at NASA Johnson Space Center's coordinate with a
icon showing its name
marker = folium.map.Marker(
    nasa_coordinate,
    # Create an icon as a text label
    icon=DivIcon(
        icon_size=(20,20),
        icon_anchor=(0,0),
        html='<div style="font-size: 12; color:#d35400;"><b>%s</b></div>' %
'NASA JSC',
        )
    )
site_map.add_child(circle)
site_map.add_child(marker)


#`folium.Circle(coordinate, radius=1000, color='#000000',
#fill=True).add_child(folium.Popup(...))`

#import folium
#from folium.features import DivIcon

# Initialisation de la carte (déjà créée dans votre notebook)
# site_map = folium.Map(location=nasa_coordinate, zoom_start=4)
```

```
# TODO: Create and add folium.Circle and folium.Marker for each launch site
on the site map
for index, row in launch_sites_df.iterrows():
    # 1. Extraction des coordonnées (Latitude, Longitude) et du nom du site
    coordinate = [row['Lat'], row['Long']]
    site_name = row['Launch Site']

    # 2. Création et ajout du cercle entourant le site de lancement
    circle = folium.Circle(
        location=coordinate,
        radius=1000,
        color='#d35400', # Vous pouvez utiliser '#000000' ou la couleur
orange native du projet
        fill=True
    ).add_child(folium.Popup(site_name))

    # 3. Création et ajout du marqueur textuel (DivIcon) pour afficher le
nom du site
    marker = folium.Marker(
        location=coordinate,
        icon=DivIcon(
            icon_size=(20, 20),
            icon_anchor=(0, 0),
            html=f'<div style="font-size: 12px;
color:#d35400;"><b>{site_name}</b></div>'
        )
    )


# 4. Ajout des éléments sur la carte principale
    site_map.add_child(circle)
    site_map.add_child(marker)

# Affichage de la carte
site_map


# Coordinates for the center of the map (example: Kennedy Space Center)
nasa_coordinate = [28.5721, -80.6480]

# Initial the map
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)

# Example dataset: Launch site names and their coordinates
launch_sites = [
    {'name': 'KSC LC-39A', 'coord': [28.5721, -80.6480]},
    {'name': 'CCAFS SLC-40', 'coord': [28.5623, -80.5774]},
    {'name': 'VAFB SLC-4E', 'coord': [34.6321, -120.6106]}
]

# For each launch site, add a Circle and a Marker to the map
for site in launch_sites:
    coordinate = site['coord']
    site_name = site['name']

    # Add a Circle object based on its coordinate (Lat, Long)
    folium.Circle(
```

```
        location=coordinate,
        radius=1000, # Radius in meters
        color='#d35400',
        fill=True,
        fill_color='#d35400',
        fill_opacity=0.2
    ).add_to(site_map)
# Add Launch site name as a custom Marker with DivIcon
    folium.Marker(
        location=coordinate,
        popup=site_name,
        icon=DivIcon(
            icon_size=(20, 20),
            icon_anchor=(0, 0),
            html='<div style="font-size: 12pt;
color:#d35400;"><b>%s</b></div>' % site_name
        )
    ).add_to(site_map)

# Display the map
site_map


spacex_df.tail(10)


marker_cluster = MarkerCluster()


# Fonction pour attribuer une couleur selon le résultat du lancement
def assign_marker_color(launch_outcome):
    if launch_outcome == 1:
        return 'green'
    else:
        return 'red'

# Application de la fonction pour créer la nouvelle colonne
spacex_df['marker_color'] = spacex_df['class'].apply(assign_marker_color)


# Function to assign color to launch outcome
def assign_marker_color(launch_outcome):
    if launch_outcome == 1:
        return 'green'
    else:
        return 'red'

spacex_df['marker_color'] = spacex_df['class'].apply(assign_marker_color)
spacex_df.tail(10)


# On parcourt chaque ligne du DataFrame spacex_df
for index, record in spacex_df.iterrows():

    # 1. On crée l'objet Marker avec ses coordonnées (Latitude, Longitude)
    marker = folium.Marker(
        location=[record['Lat'], record['Long']],
```

```
        # 2. On personnalise la couleur de l'icône selon la réussite (vert)
ou l'échec (rouge)
        icon=folium.Icon(color='white', icon_color=record['marker_color']),

        # Optionnel : Ajout d'une bulle info avec le nom du site de
lancement
        popup=f"Site: {record['Launch Site']}"
    )

    # 3. On ajoute le marqueur directement au groupe marker_cluster
    marker_cluster.add_child(marker)

# 4. (Rappel) N'oubliez pas d'ajouter le cluster à votre carte principale
si ce n'est pas déjà fait
# site_map.add_child(marker_cluster)


# Add marker_cluster to current site_map
site_map.add_child(marker_cluster)

# for each row in spacex_df data frame
# create a Marker object with its coordinate
# and customize the Marker's icon property to indicate if this launch was
successed or failed,
# e.g., icon=folium.Icon(color='white', icon_color=row['marker_color']
for index, record in spacex_df.iterrows():
    # TODO: Create and add a Marker cluster to the site map
    # marker = folium.Marker(...)
    marker_cluster.add_child(marker)

site_map


# Add Mouse Position to get the coordinate (Lat, Long) for a mouse over on
the map
formatter = "function(num) {return L.Util.formatNum(num, 5);};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=20,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)

site_map.add_child(mouse_position)
site_map


from math import sin, cos, sqrt, atan2, radians

def calculate_distance(lat1, lon1, lat2, lon2):
    # approximate radius of earth in km
    R = 6373.0
```

```
    lat1 = radians(lat1)
    lon1 = radians(lon1)
    lat2 = radians(lat2)
    lon2 = radians(lon2)

    dlon = lon2 - lon1
    dlat = lat2 - lat1

    a = sin(dlat / 2)**2 + cos(lat1) * cos(lat2) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    distance = R * c
    return distance


# 1. Définir les coordonnées (Exemple avec Cap Canaveral)
launch_site_lat = 28.56319
launch_site_lon = -80.57682
coastline_lat = 28.56367
coastline_lon = -80.57163

# 2. Calculer la distance en utilisant la fonction existante
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon,
coastline_lat, coastline_lon)

# 3. Créer et ajouter un marqueur (Marker) sur le point de la côte
distance_marker = folium.Marker(
    [coastline_lat, coastline_lon],
    icon=DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12pt;
color:#d35400;"><b>{distance_coastline:.2f} KM</b></div>',
    )
)
site_map.add_child(distance_marker)

# 4. Tracer une ligne (PolyLine) entre le site de lancement et la côte
lines = folium.PolyLine(
    locations=[[launch_site_lat, launch_site_lon], [coastline_lat,
coastline_lon]],
    weight=1
)
site_map.add_child(lines)



import math
#import folium
from folium.plugins import MousePosition

# --- Fonction pour calculer la distance (Formule de Haversine) ---
def calculate_distance(lat1, lon1, lat2, lon2):
    # Rayon de la Terre en kilomètres
    R = 6373.0
```

```
    lat1_rad = math.radians(lat1)
    lon1_rad = math.radians(lon1)
    lat2_rad = math.radians(lat2)
    lon2_rad = math.radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad

    a = math.sin(dlat / 2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) *
math.sin(dlon / 2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

    distance = R * c
    return distance
# --- Configuration initiale de la carte ---
# Coordonnées du site de lancement (Exemple : Cape Canaveral)
launch_site_lat = 28.56319
launch_site_lon = -80.57683

site_map = folium.Map(location=[launch_site_lat, launch_site_lon],
zoom_start=12)

# Ajouter l'outil MousePosition pour trouver facilement les coordonnées au
survol
formatter = "function(num) {return L.Util.formatNum(num, 5);};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=20,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)
site_map.add_child(mouse_position)

# --- Point de lancement : Marqueur ---
folium.Marker(
    [launch_site_lat, launch_site_lon],
    popup='Site de lancement',
    icon=folium.Icon(color='blue', icon='rocket')
).add_to(site_map)

# ========================================
# 1. TODO: Définir la coordonnée de la côte trouvée et calculer la distance
# ========================================
# Coordonnées de la côte la plus proche trouvée via MousePosition (Exemple)
coastline_lat = 28.56367
coastline_lon = -80.57163

# Calcul de la distance
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon,
coastline_lat, coastline_lon)


# ========================================
```

```
# 2. TODO: Créer un folium.Marker pour afficher la distance
# =========================================
# Créer le marqueur avec une icône personnalisée (DivIcon) pour afficher du
texte HTML
distance_marker = folium.Marker(
    [coastline_lat, coastline_lon],
    icon=folium.DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html='<div style="font-size: 12pt; color: #d35400; font-weight:
bold;"><b>{:10.2f} km</b></div>'.format(distance_coastline),
    )
)
site_map.add_child(distance_marker)


# =========================================
# 3. BONUS: Tracer une ligne entre le site de lancement et la côte
# =========================================
coordinates = [[launch_site_lat, launch_site_lon], [coastline_lat,
coastline_lon]]
lines = folium.PolyLine(locations=coordinates, weight=2, color='red')
site_map.add_child(lines)

# Afficher la carte (dans un notebook Jupyter)
# site_map


#import folium
from math import sin, cos, sqrt, atan2, radians

# Fonction de calcul de distance (formule de Haversine)
def calculate_distance(lat1, lon1, lat2, lon2):
    # Rayon de la Terre en kilomètres
    R = 6373.0

    lat1_rad = radians(lat1)
    lon1_rad = radians(lon1)
    lat2_rad = radians(lat2)
    lon2_rad = radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad

    a = sin(dlat / 2)**2 + cos(lat1_rad) * cos(lat2_rad) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    distance = R * c
    return distance

# --- Configuration initiale ---
# Exemple de coordonnées (Launch site : Cape Canaveral LC-39A)
launch_site_lat = 28.57325
launch_site_lon = -80.64689

# 1. TODO: Coordonnées de la côte la plus proche identifiées avec
MousePosition
```

```
coastline_lat = 28.56367
coastline_lon = -80.57163


# Calcul de la distance
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon,
coastline_lat, coastline_lon)

# Création de la carte Folium centrée sur le site de lancement
site_map = folium.Map(location=[launch_site_lat, launch_site_lon],
zoom_start=12)


# 2. TODO: Créer un folium.Marker personnalisé pour afficher la distance
distance_marker = folium.Marker(
    [coastline_lat, coastline_lon],
    icon=folium.DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12pt;
color:#d35400;"><b>{distance_coastline:.2f} KM</b></div>',
    )
)
site_map.add_child(distance_marker)

# 3. TODO: Dessiner une PolyLine entre le site de lancement et le point
côtier
coordinates = [[launch_site_lat, launch_site_lon], [coastline_lat,
coastline_lon]]
lines = folium.PolyLine(
    locations=coordinates,
    weight=1,
    color='blue'
)
#site_map.add_child(lines)

# Afficher la carte (dans un notebook Jupyter)
# site_map


# Create a `folium.PolyLine` object using the coastline coordinates and
launch site coordinate
# lines=folium.PolyLine(locations=coordinates, weight=1)
site_map.add_child(lines)



import folium
from folium.plugins import MousePosition
import math

# 1. Fonction pour calculer la distance (Formule de Haversine)
def calculate_distance(lat1, lon1, lat2, lon2):
    # Rayon de la Terre en kilomètres
    R = 6373.0

    lat1_rad = math.radians(lat1)
```

```
    lon1_rad = math.radians(lon1)
    lat2_rad = math.radians(lat2)
    lon2_rad = math.radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad

    a = math.sin(dlat / 2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) *
math.sin(dlon / 2)**2
    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

    return R * c
# Initialisation de la carte et du MousePosition pour trouver les
coordonnées
launch_site_lat, launch_site_lon = 28.56319, -80.57682 # Exemple : CCAFS
SLC-40
site_map = folium.Map(location=[launch_site_lat, launch_site_lon],
zoom_start=10)

formatter = "function(num) {return L.Util.formatNum(num, 5);};"
mouse_position = MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=20,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)
site_map.add_child(mouse_position)
# ============================================================================
# TODO 1 & 2 : Trouver la côte, calculer la distance et ajouter le Marker
# ============================================================================

# Coordonnées trouvées grâce à MousePosition sur le littoral le plus proche
coastline_lat, coastline_lon = 28.56367, -80.57163

# Calcul de la distance
distance_coastline = calculate_distance(launch_site_lat, launch_site_lon,
coastline_lat, coastline_lon)

# Création du marqueur avec une icône personnalisée affichant la distance
distance_marker = folium.Marker(
    [coastline_lat, coastline_lon],
    icon=folium.DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12pt;
color:#d35400;"><b>{distance_coastline:.2f} KM</b></div>',
    )
)
site_map.add_child(distance_marker)
# Tracer la ligne (PolyLine) entre le site et la côte
lines = folium.PolyLine(
    locations=[[launch_site_lat, launch_site_lon], [coastline_lat,
coastline_lon]],
```

```
    weight=1
)
site_map.add_child(lines)


# ============================================================================
# TODO 4 : Répéter l'opération pour une ville, une autoroute ou voie ferrée
# ============================================================================

# Exemple avec la ville la plus proche (Titusville)
city_lat, city_lon = 28.61200, -80.80700
distance_city = calculate_distance(launch_site_lat, launch_site_lon,
city_lat, city_lon)

# Marqueur pour la ville
city_marker = folium.Marker(
    [city_lat, city_lon],
    icon=folium.DivIcon(
        icon_size=(20, 20),
        icon_anchor=(0, 0),
        html=f'<div style="font-size: 12pt;
color:#2980b9;"><b>{distance_city:.2f} KM</b></div>',
    )
)
site_map.add_child(city_marker)
# Ligne vers la ville
lines_city = folium.PolyLine(
    locations=[[launch_site_lat, launch_site_lon], [city_lat, city_lon]],
    weight=1,
    color='blue'
)
site_map.add_child(lines_city)

# Afficher la carte (si vous êtes sur un notebook Jupyter)
# site_map



#import folium
from math import sin, cos, sqrt, atan2, radians

# ----------------------------------------------------
# 1. FONCTION DE CALCUL DE DISTANCE (Formule de Haversine)
# ----------------------------------------------------
def calculate_distance(lat1, lon1, lat2, lon2):
    # Rayon de la Terre en kilomètres
    R = 6373.0

    lat1_rad = radians(lat1)
    lon1_rad = radians(lon1)
    lat2_rad = radians(lat2)
    lon2_rad = radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad
```

```
    a = sin(dlat / 2)**2 + cos(lat1_rad) * cos(lat2_rad) * sin(dlon / 2)**2
    c = 2 * atan2(sqrt(a), sqrt(1 - a))

    return R * c
# -----------------------------------------------------
# 2. CONFIGURATION INITIALE DE LA CARTE & DU SITE
# -----------------------------------------------------
# Exemple avec le site de lancement Cape Canaveral (LC-39A)
launch_site_lat = 28.57325
launch_site_lon = -80.64689

# Création de la carte de base
site_map = folium.Map(location=[launch_site_lat, launch_site_lon],
zoom_start=12)

# Ajout du plugin MousePosition pour trouver les coordonnées au survol
formatter = "function(num) {return L.Util.formatNum(num, 5);};"
mouse_position = folium.plugins.MousePosition(
    position='topright',
    separator=' Long: ',
    empty_string='NaN',
    lng_first=False,
    num_digits=20,
    prefix='Lat:',
    lat_formatter=formatter,
    lng_formatter=formatter,
)
site_map.add_child(mouse_position)
# -----------------------------------------------------
# 3. COORDONNÉES DES POINTS LES PLUS PROCHES (À trouver via MousePosition)
# -----------------------------------------------------
# Remplacez ces coordonnées fictives par celles trouvées avec votre souris
:
coordinates_poi = {
    "coastline": {"lat": 28.56367, "lon": -80.57163, "color": "blue"},
    "city":      {"lat": 28.61200, "lon": -80.80700, "color": "green"},
    "railway":   {"lat": 28.57245, "lon": -80.58528, "color": "orange"},
    "highway":   {"lat": 28.57304, "lon": -80.65545, "color": "red"}
}

# -----------------------------------------------------
# 4. CRÉATION DES MARQUEURS ET DES LIGNES (POLY_LINES)
# -----------------------------------------------------
for poi_name, coords in coordinates_poi.items():
    # Calcul de la distance entre le site et le POI
    distance = calculate_distance(launch_site_lat, launch_site_lon,
coords["lat"], coords["lon"])

    # TODO 2 & 5: Créer un folium.Marker pour afficher la distance
    distance_marker = folium.Marker(
        [coords["lat"], coords["lon"]],
        icon=folium.DivIcon(
            icon_size=(20, 20),
            icon_anchor=(0, 0),
            html=f'<div style="font-size: 12pt;
color:{coords["color"]};"><b>{distance:.2f} KM</b></div>',
        )
```

```
    )
    site_map.add_child(distance_marker)
# TODO 4: Tracer une ligne (PolyLine) entre le site de lancement et le POI
    line_coordinates = [[launch_site_lat, launch_site_lon], [coords["lat"],
coords["lon"]]]
    lines = folium.PolyLine(
        locations=line_coordinates,
        weight=2,
        color=coords["color"]
    )
    site_map.add_child(lines)

# Affichage ou sauvegarde de la carte
# site_map.save("launch_site_distances.html")
site_map
```

Laboratoire pratique : Construire un tableau de bord interactif avec Plotly Dash

```
# Import required libraries

import pandas as pd

import dash

import dash_html_components as html

import dash_core_components as dcc

from dash.dependencies import Input, Output

import plotly.express as px


# Read the airline data into pandas dataframe

spacex_df = pd.read_csv("spacex_launch_dash.csv")

max_payload = spacex_df['Payload Mass (kg)'].max()

min_payload = spacex_df['Payload Mass (kg)'].min()


# Create a dash application

app = dash.Dash(__name__)


# Create an app layout

app.layout = html.Div(children=[html.H1('SpaceX Launch Records Dashboard',

                                        style={'textAlign': 'center', 'color': '#503D36',

                                               'font-size': 40}),
```

```
# TASK 1: Add a dropdown list to enable Launch Site selection
# The default select value is for ALL sites
dcc.Dropdown(id='site-dropdown',
        options=[
            {'label': 'All Sites', 'value': 'ALL'},
            {'label': 'CCAFS LC-40', 'value': 'CCAFS LC-40'},
            {'label': 'VAFB SLC-4E', 'value': 'VAFB SLC-4E'},
            {'label': 'KSC LC-39A', 'value': 'KSC LC-39A'},
            {'label': 'CCAFS SLC-40', 'value': 'CCAFS SLC-40'}
            ],
        value='ALL',
        placeholder='Select a Launch Site here',
        searchable=True
        ),
html.Br(),

# TASK 2: Add a pie chart to show the total successful launches count for all sites
# If a specific launch site was selected, show the Success vs. Failed counts for the
site

html.Div(dcc.Graph(id='success-pie-chart')),
html.Br(),

html.P("Payload range (Kg):"),
# TASK 3: Add a slider to select payload range
#dcc.RangeSlider(id='payload-slider',...)
dcc.RangeSlider(id='payload-slider',
          min=0,
          max=10000,
          step=1000,
          value=[min_payload, max_payload]
          ),
```

```
                    # TASK 4: Add a scatter chart to show the correlation between payload and launch
success
                    html.Div(dcc.Graph(id='success-payload-scatter-chart')),
                    ])


# TASK 2:
# Add a callback function for `site-dropdown` as input, `success-pie-chart` as output
@app.callback(Output(component_id='success-pie-chart', component_property='figure'),
          Input(component_id='site-dropdown', component_property='value'))
def get_pie_chart(entered_site):
   filtered_df = spacex_df
   if entered_site == 'ALL':
      fig = px.pie(filtered_df, values='class',
      names='Launch Site',
      title='Success Count for all launch sites')
      return fig
   else:
      # return the outcomes piechart for a selected site
      filtered_df=spacex_df[spacex_df['Launch Site']== entered_site]
      filtered_df=filtered_df.groupby(['Launch Site','class']).size().reset_index(name='class count')
      fig=px.pie(filtered_df,values='class count',names='class',title=f"Total Success Launches for site
{entered_site}")
      return fig



# TASK 4:
# Add a callback function for `site-dropdown` and `payload-slider` as inputs, `success-payload-
scatter-chart` as output
@app.callback(Output(component_id='success-payload-scatter-
chart',component_property='figure'),
            [Input(component_id='site-dropdown',component_property='value'),
```

```
            Input(component_id='payload-slider',component_property='value')])

def scatter(entered_site,payload):
    filtered_df = spacex_df[spacex_df['Payload Mass (kg)'].between(payload[0],payload[1])]
    # thought reusing filtered_df may cause issues, but tried it out of curiosity and it seems to be working fine


    if entered_site=='ALL':
        fig=px.scatter(filtered_df,x='Payload Mass (kg)',y='class',color='Booster Version Category',title='Success count on Payload mass for all sites')
        return fig
    else:
        fig=px.scatter(filtered_df[filtered_df['Launch Site']==entered_site],x='Payload Mass (kg)',y='class',color='Booster Version Category',title=f"Success count on Payload mass for site {entered_site}")
        return fig


# Run the app
if __name__ == '__main__':
    app.run_server()
```

Hands on Lab: Complete the Machine Learning Prediction lab

```
!pip install numpy
!pip install pandas
!pip install seaborn
!pip install scikit-learn
```

```
# Pandas is a software library written for the Python programming language for data manipulation and analysis.
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays
import numpy as np
# Matplotlib is a plotting library for python and pyplot gives us a MatLab like plotting framework. We will use this in our plotter function to plot data.
```

```
import matplotlib.pyplot as plt
#Seaborn is a Python data visualization library based on matplotlib. It
provides a high-level interface for drawing attractive and informative
statistical graphics
import seaborn as sns
# Preprocessing allows us to standarsize our data
from sklearn import preprocessing
# Allows us to split our data into training and testing data
from sklearn.model_selection import train_test_split
# Allows us to test parameters of classification algorithms and find the
best one
from sklearn.model_selection import GridSearchCV
# Logistic Regression classification algorithm
from sklearn.linear_model import LogisticRegression
# Support Vector Machine classification algorithm
from sklearn.svm import SVC
# Decision Tree classification algorithm
from sklearn.tree import DecisionTreeClassifier
# K Nearest Neighbors classification algorithm
from sklearn.neighbors import KNeighborsClassifier


def plot_confusion_matrix(y,y_predict):
    "this function plots the confusion matrix"
    from sklearn.metrics import confusion_matrix

    cm = confusion_matrix(y, y_predict)
    ax= plt.subplot()
    sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
    ax.set_xlabel('Predicted labels')
    ax.set_ylabel('True labels')
    ax.set_title('Confusion Matrix');
    ax.xaxis.set_ticklabels(['did not land', 'land']);
ax.yaxis.set_ticklabels(['did not land', 'landed'])
    plt.show()


data = pd.read_csv("https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/dataset_part_2.csv")


data.head()


X = pd.read_csv('https://cf-courses-data.s3.us.cloud-object-
storage.appdomain.cloud/IBM-DS0321EN-
SkillsNetwork/datasets/dataset_part_3.csv')


X.head(100)


Y = data['Class'].to_numpy()


# students get this
#transform = preprocessing.StandardScaler()
```

```
# students get this
transform = preprocessing.StandardScaler()

# Complete TASK 2 here:
X = transform.fit_transform(X)


X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2,
random_state=2)


Y_test.shape


# Provided hyperparameter dictionary
parameters ={'C':[0.01,0.1,1],
             'penalty':['l2'],
             'solver':['lbfgs']}


# 1. Create a logistic regression object
parameters ={"C":[0.01,0.1,1],'penalty':['l2'], 'solver':['lbfgs']}# l1
lasso l2 ridge
lr=LogisticRegression()


# 2. Create a GridSearchCV object with cv = 10
logreg_cv = GridSearchCV(estimator=lr, param_grid=parameters, cv=10)

# 3. Fit the object to find the best parameters
logreg_cv.fit(X_train, Y_train)


print("tuned hpyerparameters :(best parameters) ",logreg_cv.best_params_)
print("accuracy :",logreg_cv.best_score_)


# Calculate and print the accuracy score
test_accuracy = logreg_cv.score(X_test, Y_test)
print(f"Test Accuracy: {test_accuracy}")


yhat=logreg_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)


# Provided dictionary of parameters
awaitparameters = {'kernel':('linear', 'rbf','poly','rbf', 'sigmoid'),
             'C': np.logspace(-3, 3, 5),
             'gamma':np.logspace(-3, 3, 5)}
# 1. Create a support vector machine object
boolsvm = SVC()


# 2. Create a GridSearchCV object svm_cv with cv = 10
svm_cv = GridSearchCV(estimator=boolsvm, param_grid=parameters, cv=10)
```

```
# 3. Fit the object to find the best parameters
svm_cv.fit(X_train, Y_train)


# Calculate accuracy on the test data
accuracy = logreg_cv.score(X_test, Y_test)

# Print the result
print(f"Test Accuracy: {accuracy * 100:.2f}%")


yhat=svm_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)


parameters = {'criterion': ['gini', 'entropy'],
     'splitter': ['best', 'random'],
     'max_depth': [2*n for n in range(1,10)],
     'max_features': ['auto', 'sqrt'],
     'min_samples_leaf': [1, 2, 4],
     'min_samples_split': [2, 5, 10]}

# 1. Create a decision tree classifier object
tree = DecisionTreeClassifier()


# 2. Create a GridSearchCV object tree_cv with 10-fold cross-validation
tree_cv = GridSearchCV(estimator=tree, param_grid=parameters, cv=10)

# 3. Fit the object to find the best hyperparameters
tree_cv.fit(X_train, Y_train)


print("tuned hpyerparameters :(best parameters) ",tree_cv.best_params_)
print("accuracy :",tree_cv.best_score_)


# Calculate the accuracy on the test data
tree_cv_score = tree_cv.score(X_test, Y_test)
print("Decision Tree - Accuracy using method score:", tree_cv_score)


yhat = tree_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)


# Define the parameters dictionary (example values provided)
parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
              'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
              'p': [1,2]}
# Create a k-nearest neighbors object
KNN = KNeighborsClassifier()


# Create a GridSearchCV object with 10-fold cross-validation
knn_cv = GridSearchCV(estimator=KNN , param_grid=parameters, cv=10)

# Fit the object to find the best parameters
```

```
# Replace X_train and Y_train with your actual training data variables
knn_cv.fit(X_train, Y_train)


print("tuned hpyerparameters :(best parameters) ",knn_cv.best_params_)
print("accuracy :",knn_cv.best_score_)


knn_cv.score(X_test, Y_test)


yhat = knn_cv.predict(X_test)
plot_confusion_matrix(Y_test,yhat)
```

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns

# Assuming X_train, X_test, Y_train, Y_test are predefined
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "SVC": SVC(),
    "Decision Tree": DecisionTreeClassifier()
}

# Evaluate models
results = []
for name, model in models.items():
    model.fit(X_train, Y_train)
    acc = accuracy_score(Y_test, model.predict(X_test))
    results.append({"Method": name, "Accuracy": acc})

# Sort and display results
df_results = pd.DataFrame(results).sort_values(by="Accuracy",
ascending=False)
print(df_results)

# Visualize results
sns.barplot(x="Accuracy", y="Method", data=df_results)
plt.title("Model Performance Comparison")
plt.show()
# Print best method
print(f"Best method: {df_results.iloc[0]['Method']}")
```